

Содержание

- 1 Конкурентность и Параллелизм
- 2 Разделяемая Память и Передача Сообщений
- 3 Атомарность, Гонки, Дедлоки и Голодание
- 4 Горутины (Goroutines)
- 5 Каналы (Channels)
- 6 Select
- 7 Мьютексы (Mutex)
- 8 WaitGroup
- 9 Condition Variables
- 10 Итоги

Ключевое различие

- **Конкурентность** — способ организации программы, работа с несколькими задачами одновременно
- **Параллелизм** — одновременное выполнение задач

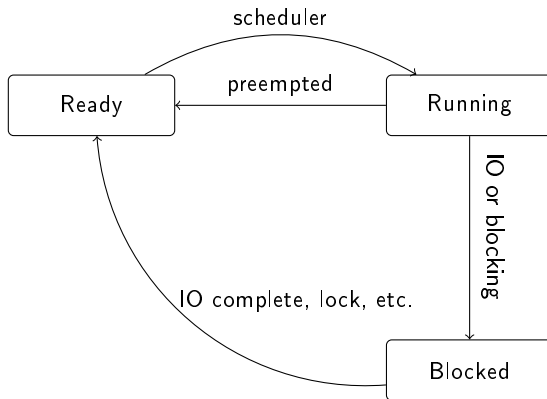
*“Concurrency is about dealing with lots of things at once.
Parallelism is about doing lots of things at once.” — Rob Pike*

Определение

Конкурентность — это способность различных частей программы выполняться **вне порядка (out-of-order)** или в частичном порядке без влияния на результат.

Конкурентность — это **модель программирования**, как ООП или функциональное программирование, которая разделяет проблему на временные компоненты.

Состояния Потока (Горутины)



- **Running:** Выполняется на процессоре
- **Ready:** Готова к выполнению, ожидает планировщик
- **Blocked:** Ожидает ввода-вывода, мьютекса или канала

Кооперативная vs Вытесняющая Многозадачность

- **Ранние версии Go (< 1.14):** Кооперативный планировщик. Горутина могла заблокировать другие, если не отдавала управление.
- **Современные версии Go:** Вытесняющий планировщик. Runtime может прервать долго работающую горутину.

Важно для анализа

Понимание планировщика — ключ к выявлению возможных вариантов поведения конкурентной системы.

Разделяемая Память

- Несколько потоков используют одну память
- Требуется синхронизация (мьютексы)
- Нужны барьеры памяти
- Риск data races

Передача Сообщений (CSP)

- Горутины обмениваются сообщениями
- Отсутствие общих данных
- Естественная синхронизация
- Основа Go (каналы)

Проблема Переупорядочивания

```
var locked, y int
func f() {
    locked = 1
    y = 2
}
```

Без синхронизации, снимок памяти может быть любым:

- `locked=0, y=0` (операции не записаны в память)
- `locked=0, y=2` (`y` обновлен, `locked` — нет)
- `locked=1, y=0` (наоборот)
- `locked=1, y=2` (оба обновлены)

“Do not communicate by sharing memory; instead, share memory by communicating.”

**Не общайтесь через разделяемую память; вместо этого
разделяйте память через общение.**

Состояние гонки (Race Condition)

Проблема банковского счета

Две горутины снимают деньги одновременно:

- Исходный баланс: 10
- Goroutine 1: снимает 6
- Goroutine 2: снимает 5

Ожидаемый результат: одна операция успешна, другая — нет.

Возможный сценарий

Обе горутины проверяют баланс (10), видят достаточно средств и снимают деньги. Итог: баланс = -1.

Гонка данных (Data Race)

Условия гонки данных:

- 1 Два или более потоков обращаются к одной ячейке памяти
- 2 Хотя бы один поток выполняет запись
- 3 Нет синхронизации для упорядочивания операций

Важно

Гонка данных — это частный случай состояния гонки, который приводит к неопределенному поведению и может быть очень трудно воспроизводим.

Исправление гонки: Мьютекс

```
type Account struct {  
    sync.Mutex  
    Balance int  
}  
  
func (act *Account) Withdraw(amt int) error {  
    act.Lock()  
    defer act.Unlock()  
    if act.Balance < amt {  
        return ErrInsufficientFunds  
    }  
    act.Balance -= amt  
    return nil  
}
```

Дедлок (Deadlock)

Условия Коффмана:

- 1 Взаимное исключение
- 2 Удержание и ожидание
- 3 Отсутствие принудительного освобождения
- 4 Циклическое ожидание

В Golang есть и другие дедлоки, не вписывающиеся в условия для CSP (чтение из канала, в который никто ничего не посылает, или запись в небуферизованный канал, из которого никто не читает, повторное закрытие мьютекса, чтение из nil-канала).

```
1 func Transfer(from, to *Account, amt int) {  
2     from.Lock()  
3     to.Lock()    // Блокировка  
4     // ...  
5 }  
6 // Пример циклического ожидания  
7 // G1: lock A, ждет B  
8 // G2: lock B, ждет A
```

Предотвращение Дедлока

```
func Transfer(from, to *Account, amt int) error {  
    if from.ID < to.ID {  
        from.Lock()  
        to.Lock()  
    } else {  
        to.Lock()  
        from.Lock()  
    }  
    defer from.Unlock()  
    defer to.Unlock()  
    // ... операция перевода  
}
```

Чтобы исключить циклическое ожидание, конкурентные задачи, разделяющие общий ресурс, следует частично упорядочивать.

Голодание (Starvation)

- Ситуация, когда процесс длительное время или бесконечно не получает доступа к ресурсу
- Результат примитивных алгоритмов планирования или ошибок в коде
- Может быть вызвано "жадной" горутинной, долго удерживающей мьютекс

Livelock

Особая форма голодания: процессы работают, но не делают полезной работы. Например, два человека в узком коридоре, пытающиеся уступить друг другу дорогу.

Свойства Безопасности (Safety)

- Плохое никогда не случается
- Отсутствие дедлока
- Взаимное исключение
- Корректность данных

Свойства Живучести (Liveness)

- Хорошее в итоге случается
- Завершение программы
- Ответ на запрос
- Вход в критическую секцию

Livelock

```
go func() {  
    for {  
        if mutex1.TryLock() {  
            if mutex2.TryLock() {  
                // ...  
                mutex2.Unlock()  
            }  
            mutex1.Unlock()  
        }  
    }  
}()
```

```
1  go func() {  
2      for {  
3          if mutex2.TryLock() {  
4              if mutex1.TryLock() {  
5                  // ...  
6                  mutex1.Unlock()  
7              }  
8              mutex2.Unlock()  
9          }  
10     }  
11 }()
```


- Конкурентность - не Параллелизм. Конкурентность — это модель, параллелизм — выполнение.
- Go поддерживает обе модели: разделяемую память (мьютексы) и передачу сообщений (каналы).
- Гонки данных возникают при одновременном доступе к памяти без синхронизации.
- Дедлоки требуют осторожного подхода к блокировкам (упорядочивание).
- **Главное правило:** Корректность важнее производительности.

Потоки ОС

- Управляются ОС
- Большой стек (МБ)
- Дорогое создание
- Переключение контекста — ОС

Горутины

- Управляются runtime Go
- Маленький стартовый стек (2КБ, растет)
- Легкое создание
- М:N планировщик (М горутин на N потоков)

GOMAXPROCS

Количество потоков ОС, выполняющих Go-код. По умолчанию = количество ядер CPU.

Создание горутин

// Базовый синтаксис

```
go f()  
go g(i, j)
```

// Анонимная функция

```
go func() {  
    fmt.Println("hello")  
}()
```

// С параметрами

```
go func(i, j int) {  
    fmt.Println(i + j)  
}(1, 2)
```

Параметры вычисляются **до** запуска горутин и передаются копией.

Escape Analysis

```
func main() {  
    s := "hello"  
    go func() {  
        fmt.Println(s) // s "escape" в heap  
    }()  
    time.Sleep(100 * time.Millisecond)  
}
```

Компилятор определяет, что переменная может использоваться после возврата из функции (горутина живет дольше), и выделяет память в heap вместо stack.

Остановка горутин

Горутину нельзя принудительно остановить извне. Только кооперативно:

- Через канал (signal)
- Через разделяемую переменную (с атомиком/мьютексом)
- Завершение `main()` убивает все горутины

```
done := make(chan struct{})
go func() {
    for {
        select {
        case <-done:
            return // Чистый выход
        default:
            // работа
        }
    }
}()
close(done) // Сигнал остановки
```

Каналы: Базовый синтаксис

```
// Создание
ch := make(chan int)      // Небуферизированный
ch := make(chan int, 10)  // Буферизированный (cap=10)

// Отправка
ch <- 42

// Получение
x := <-ch
x, ok := <-ch  // ok == false если канал закрыт

// Направленные каналы
var sendOnly chan<- int  // Только отправка
var recvOnly <-chan int  // Только получение

// Длина и емкость
len(ch)  // Количество элементов в буфере
cap(ch)  // Емкость буфера
```

Небуферизированные каналы

```
ch := make(chan int)
```

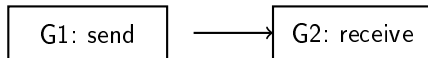
```
// Goroutine 1
```

```
ch <- 42 // Блокируется, пока  
        // нет получателя
```

```
// Goroutine 2
```

```
x := <-ch // Блокируется, пока  
          // нет отправителя
```

Небуферизированный канал — это точка синхронизации. Отправка и получение должны произойти **одновременно**.



Буферизированные каналы

```
ch := make(chan int, 2) // Емкость = 2
```

```
ch <- 1 // Не блокируется (буфер 1/2)
```

```
ch <- 2 // Не блокируется (буфер 2/2)
```

```
ch <- 3 // БЛОКИРУЕТСЯ! Буфер полон
```

```
x := <-ch // Освобождает место, можно отправить 3
```

Гарантии памяти: для буферизированного канала с емкостью C , n -е получение происходит после завершения $(n + C)$ -й отправки.

Заккрытие каналов

```
close(ch) // Закреть канал

// Получение из закрытого канала
x, ok := <-ch
// ok == false, x == zero value

// Отправка в закрытый канал
ch <- 42 // PANIC

// Повторное закрытие
close(ch) // PANIC
```

Идиома: Broadcast через close

```
done := make(chan struct{})
close(done) // Все получатели увидят, что канал закрыт
```

Заккрытие канала — способ уведомить множество горутин одновременно.

Range по каналу

```
ch := make(chan int)

go func() {
    for i := 0; i < 5; i++ {
        ch <- i
    }
    close(ch) // Обязательно закрыть!
}()

// Итерируемся, пока канал не закроется
for v := range ch {
    fmt.Println(v) // 0, 1, 2, 3, 4
}
```

Без 'close(ch)' range будет ждать вечно.

Передача владения через каналы

```
type Data struct {  
    Values map[string]interface{}  
}  
  
func processData(data Data, pipeline chan Data) {  
    data.Values = getInitialValues()  
    pipeline <- data // Отправляем копию  
    // data.Values["status"] = "sent" // ОПАСНО - data race  
}
```

Map — это указатель на структуру. Канал передает копию указателя. Отправитель и получатель работают с **одной** map → race condition.

Хороший тон

После отправки значения через канал — **не используйте** его снова (или синхронизируйте доступ).

Select — мультиплексирование каналов

```
select {  
case x := <-ch1:  
    fmt.Println("Received from ch1:", x)  
case y := <-ch2:  
    fmt.Println("Received from ch2:", y)  
case ch3 <- z:  
    fmt.Println("Sent to ch3")  
default:  
    fmt.Println("No channel ready")  
}
```

- Выбирает **один** случайный готовый канал
- Если несколько готовы — выбирается случайно
- Если ни один не готов — default (если есть) или блокировка

Non-blocking операции

```
// Non-blocking send
select {
case ch <- x:
    sent = true
default:
    sent = false
}

// Non-blocking receive
select {
case x = <-ch:
    received = true
default:
    received = false
}
```

"Non-blocking" означает, что операция не блокируется **на момент проверки**. К моменту выполнения default канал может стать готовым.

Тонкости select (1)

```
func main() {  
    var i int  
    f := func() int {  
        i++  
        return i  
    }  
    ch1 := make(chan int)  
    ch2 := make(chan int)  
    select {  
    case ch1 <- f(): // f() ВЫЗЫВАЕТСЯ!  
    case ch2 <- f(): // f() ВЫЗЫВАЕТСЯ!  
    default:  
    }  
    fmt.Println(i) // Выведет 2  
}
```

Аргументы канальных операций вычисляются **до** выбора case.

Тонкости select (2)

```
func main() {  
    ch1 := make(chan int)  
    ch2 := make(chan int)  
    go func() { ch2 <- 1 }()  
    go func() { fmt.Println(<-ch1) }()  
  
    select {  
    case ch1 <- <-ch2: // Сначала <-ch2, потом ch1<-  
        time.Sleep(time.Second)  
    default:  
    }  
}
```

1. Вычисляется ' $\leftarrow$ ch2' (блокируется, пока нет значения) 2. После получения значения, проверяется готовность 'ch1' 3. Если 'ch1' не готов — выбирается 'default'

sync.Mutex (Mutual Exclusion) — базовое использование

```
type Counter struct {  
    mu    sync.Mutex  
    value int  
}  
  
func (c *Counter) Inc() {  
    c.mu.Lock()  
    defer c.mu.Unlock()  
    c.value++  
}  
  
func (c *Counter) Value() int {  
    c.mu.Lock()  
    defer c.mu.Unlock()  
    return c.value  
}
```

Мьютекс **нельзя копировать!** Используйте указатель.

Опасность копирования мьютекса

```
type Cache struct {  
    mu sync.Mutex  
    m  map[string]*Data  
}  
  
// Копирует mutex  
func (c Cache) Get(id string) (*Data, bool) {  
    c.mu.Lock() // Работает с копией  
    defer c.mu.Unlock()  
    data, ok := c.m[id]  
    return data, ok  
}
```

Почему это опасно?

Каждый вызов 'Get()' получает **собственную копию** мьютекса. Разные горуты блокируют разные мьютексы → mutual exclusion не работает

Использовать **указатель**: 'func (c *Cache) Get(...)'

Повторная блокировка — deadlock

```
var mu sync.Mutex

func f() {
    mu.Lock()
    defer mu.Unlock()
    g() // Внутри g() снова Lock()
}

func g() {
    mu.Lock() // DEADLOCK! Та же горютина ждет себя
    defer mu.Unlock()
}
```

Мьютекс не отслеживает, какая горютина его заблокировала. Повторная блокировка той же горютиной → deadlock.

sync.RWMutex — читатели/писатели

```
type Cache struct {  
    mu sync.RWMutex  
    m  map[string]*Data  
}  
  
func (c *Cache) Get(id string) *Data {  
    c.mu.RLock()           // Много читателей  
    defer c.mu.RUnlock()  
    return c.m[id]  
}  
  
func (c *Cache) Set(id string, data *Data) {  
    c.mu.Lock()           // Только один писатель  
    defer c.mu.Unlock()  
    c.m[id] = data  
}
```

- 'RLock()' — не блокирует другие 'RLock()'
- 'Lock()' — блокирует всё (и RLock, и Lock)

WaitGroup — ожидание группы горутин

```
func main() {  
    var wg sync.WaitGroup  
  
    for i := 0; i < 10; i++ {  
        wg.Add(1) // Увеличиваем счетчик ДО запуска  
        go func(i int) {  
            defer wg.Done() // Уменьшаем при завершении  
            fmt.Println(i)  
        }(i)  
    }  
  
    wg.Wait() // Блокируется, пока счетчик не станет 0  
    fmt.Println("All done")  
}
```

- 'Add()' должен вызываться **до** того, как горутина может запуститься
- 'Done()' должен вызываться **обязательно** (лучше через 'defer')

WaitGroup + каналы — проблема

```
func main() {
    ch := make(chan int)
    var wg sync.WaitGroup

    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func(i int) {
            defer wg.Done()
            ch <- i // Блокируется - нет читателя!
        }(i)
    }

    wg.Wait() // DEADLOCK! Никогда не завершится
    close(ch)
}
```

Решение: читать в отдельной горутине

```
1 go func() {
2     wg.Wait()
3     close(ch)
4 }()
5 for v := range ch {
6     fmt.Println(v)
7 }
```

sync.Cond — переменная условия

- 'Wait()' — блокирует горутину до сигнала
- 'Signal()' — будит одну ожидающую горутину
- 'Broadcast()' — будит все ожидающие горутины

```
mu := sync.Mutex{}  
cond := sync.NewCond(&mu)
```

```
// Ожидание
```

```
mu.Lock()  
for !condition() {  
    cond.Wait() // Атомарно: Unlock + ожидание + Lock  
}  
mu.Unlock()
```

```
// Сигнал
```

```
mu.Lock()  
cond.Signal() // Или Broadcast()  
mu.Unlock()
```

Producer-Consumer на Cond

```
func producer(cond *sync.Cond, queue *Queue, value int) {
    cond.L.Lock()
    for !queue.Enqueue(value) { // Пока нет места
        cond.Wait()
    }
    cond.L.Unlock()
    cond.Signal() // Сигнал consumer'ам
}
```

```
func consumer(cond *sync.Cond, queue *Queue) {
    cond.L.Lock()
    var v int
    for {
        var ok bool
        if v, ok = queue.Dequeue(); ok {
            break
        }
        cond.Wait() // Нет данных
    }
    cond.L.Unlock()
    cond.Signal() // Сигнал producer'ам
}
```

Cond vs Channel

Cond легче в использовании:

- Когда нужно разбудить **всех** ожидающих (Broadcast)
- Когда состояние сложнее, чем "есть данные в очереди"
- В уже существующей shared memory архитектуре

Большинство задач на Cond проще реализуются с каналами. Cond — для ситуаций, где нужен точный контроль над блокировками.

- **Горутины** — легковесные потоки, управляемые runtime. Замыкания требуют осторожности.
- **Каналы** — инструмент синхронизации и передачи данных. Небуферизированные — синхронные, буферизированные — с очередью.
- **Select** — мультиплексирование каналов. Аргументы вычисляются заранее.
- **Mutex/RWMutex** — защита разделяемой памяти. Не копировать! Не реентерабельный.
- **WaitGroup** — ожидание завершения группы горутин. Add() до запуска.
- **Cond** — ожидание условий. Альтернатива — каналы.